

Design and Analysis of Algorithms

Week 2: Asymptotic Notation and Complexity Classes

TOAE209/TOAH209 – Foreign Trade University

Department of Technology & Data Science

Semester 2024–2025

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Why Formal Notation Matters

- We need a way to compare algorithms without tying the answer to one machine
- Constant factors depend on language, compiler, CPU, and memory layout
- Growth rate is the part that survives scale
- Asymptotic notation is the language for that growth rate

Running idea

We will keep one campus example in mind: a student-record system that must search, sort, and validate data as the number of records grows.

What the Week Covers

- 1 How to talk about growth with O , Ω , and Θ
- 2 How to prove bounds instead of guessing them
- 3 How to compare common growth classes
- 4 How to classify problems as P, NP, NP-hard, or undecidable
- 5 Why reductions matter in practice

Goal

By the end, you should be able to read an algorithm and say what dominates its cost, why, and what class of problem it belongs to.

The Running Example

Suppose the university stores registration records:

- each record has a student ID, course code, and timestamp
- staff want to search by ID
- the system may sort records by ID or time
- the system may detect duplicates or conflicts

Question

If the record count doubles, which operations get twice as slow, which get only slightly slower, and which become much worse?

Same Code, Different Scale

- 50 records: almost any solution feels fast
- 5,000 records: inefficient loops start to show
- 500,000 records: the wrong algorithm becomes unusable

Lesson

Big-O is not about tiny inputs. It is about what survives when the dataset is large enough to expose the algorithm's structure.

Input Size Is the Real Variable

- For search, n is the number of records
- For sorting, n is the number of items to order
- For graph problems, n may be vertices and m edges
- For strings, n is the length of the text or pattern

Good habit

Always ask: what exactly does n count in this problem?

A Simple Cost Model

- Count primitive operations, comparisons, and assignments
- Ignore hardware details unless they change the dominant term
- Treat memory usage separately from running time

We usually ask

How many basic steps does the algorithm perform as a function of n ?

What Asymptotics Keeps

- the highest-order term
- the structure of loops and recursion
- the difference between linear, quadratic, and exponential growth

Example

$3n^2 + 12n + 7$ behaves like n^2 for large n .

What Asymptotics Ignores

- constant multipliers
- lower-order terms
- implementation details that do not change the dominant growth rate

Important

Ignoring constants does not mean constants are useless. They still matter for practical break-even points.

A First Timing Thought Experiment

- Algorithm A: $1000n$ steps
- Algorithm B: n^2 steps

For small n

B may be faster because the constant in A is large.

For large n

A wins because linear growth beats quadratic growth.

Outline

- 1 Why This Week Matters
- 2 **Big-O**
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Big-O: Formal Definition

Definition

$f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq f(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

- Big-O is an asymptotic upper bound
- It says f does not grow faster than a constant multiple of g

Big-O: Intuition

- Think of a ceiling above the function
- The ceiling only needs to work after some threshold n_0
- What happens for tiny inputs is not important to the definition

Reading

" $f(n)$ is $O(g(n))$ " means " f grows no faster than g up to a constant factor."

Big-O Example: Linear Search

- The algorithm checks each record until it finds the target or reaches the end
- Worst case: the target is absent or last
- Number of comparisons is at most n

Conclusion

Linear search is $O(n)$.

Big-O Example: Nested Loop

```
count = 0
for i in range(n):
    for j in range(n):
        count += 1
```

- Inner statement runs $n \cdot n$ times
- Dominant term is n^2

Conclusion

The loop is $O(n^2)$.

Big-O Example: One Heavy and One Light Term

$$3n^2 + 40n + 17$$

- the quadratic term dominates
- the linear and constant terms eventually vanish in comparison

Result

$$3n^2 + 40n + 17 = O(n^2).$$

Proving an Upper Bound

To prove $f(n) = O(g(n))$:

- 1 choose a candidate $g(n)$
- 2 find constants c and n_0
- 3 show $f(n) \leq cg(n)$ for all $n \geq n_0$

Example

For $3n^2 + 40n + 17$, one valid choice is $c = 60$ and $n_0 = 1$.

Big-O Non-Example

- n^2 is not $O(n)$
- the ratio $n^2/n = n$ grows without bound
- no constant multiple of n can keep up with n^2

Takeaway

A claim can fail because the proposed ceiling is too small.

Upper Bound Versus Tight Bound

- $2n = O(n^2)$ is true
- but it is not useful
- $2n = O(n)$ is the useful statement

Rule of thumb

Use the tightest bound you can justify.

Big-O Rules You Will Use Often

- $O(f) + O(g) = O(\max(f, g))$
- $O(cf) = O(f)$ for any constant $c > 0$
- $O(f) \cdot O(g) = O(fg)$

Interpretation

When you combine parts of an algorithm, keep only the dominating term.

Logs in Big-O

- $\log_a n$ and $\log_b n$ differ only by a constant factor
- therefore they are the same asymptotic class
- logarithms grow much slower than polynomials

Useful fact

$\log^k n = o(n^\epsilon)$ for any fixed k and $\epsilon > 0$.

Big-O and Recursion Preview

- recursion often leads to recurrence relations
- each recursive call contributes a subproblem
- the total cost is the sum of all levels

Week 3 link

In divide and conquer, Big-O will be combined with recurrences.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta**
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Big-Omega: Lower Bound

Definition

$f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq c g(n) \leq f(n) \quad \text{for all } n \geq n_0.$$

- Big-Omega is a floor
- it says f grows at least as fast as a constant multiple of g

Big-Theta: Tight Bound

Definition

$f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

- Theta means both an upper and a lower bound
- It is the most informative asymptotic statement when it is available

Theta Example

$$2n^2 + 3n + 1$$

- it is at most a constant multiple of n^2
- it is also at least a constant multiple of n^2

Conclusion

$$2n^2 + 3n + 1 = \Theta(n^2).$$

Theta Example: Linear Search

- best case: the target is first
- worst case: the target is absent or last
- in the worst case, the number of checks grows linearly

Worst-case cost

Linear search is $\Theta(n)$ in the worst case.

Theta Versus Big-O

- $O(g)$ alone only says "no worse than"
- $\Theta(g)$ says "same growth rate up to constants"
- if you know both an upper and a lower bound, report Θ

Example

n is $O(n^2)$, but only $\Theta(n)$ gives the right story.

Little-o and Little-omega

Little-o

$f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

Little-omega

$f(n) = \omega(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

Limit Test Examples

- $n = o(n^2)$ because $n/n^2 = 1/n \rightarrow 0$
- $n \log n = o(n^{1.5})$ because the polynomial wins
- $n^2 = \omega(n \log n)$ because the ratio diverges

Use carefully

Limits are a fast proof tool when the algebra is clean.

A Useful Identity

$$\log_a n = \frac{\log_b n}{\log_b a}$$

- changing the log base only multiplies by a constant
- Big-O ignores that constant

Conclusion

All log bases are equivalent asymptotically.

Theta From Two Sides

To prove $f(n) = \Theta(g(n))$:

- 1 prove $f(n) = O(g(n))$
- 2 prove $f(n) = \Omega(g(n))$

Example strategy

For a polynomial, bound by the highest-order term from above and below.

Common Misreadings

- $O(n^2)$ does not mean exactly quadratic
- $\Theta(n)$ does not mean one step per input item in code
- asymptotic notation describes growth, not line-by-line syntax

Warning

A good proof uses the definition, not intuition alone.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit**
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

How to Prove an Asymptotic Bound

- isolate the dominant term
- compare everything to a known function
- choose constants explicitly
- state the threshold n_0

Template

"For all $n \geq n_0$, we have $f(n) \leq cg(n)$."

Bounding by the Largest Term

$$5n^3 + 100n^2 + n + 1000$$

- the cubic term dominates
- the others are lower-order

Result

This expression is $\Theta(n^3)$.

Geometric Series Pattern

$$1 + 2 + 4 + \dots + 2^k$$

- the last term dominates the sum
- the total is less than twice the last term

Typical conclusion

Geometric sums are the same order as their largest term.

A Loop Becomes a Sum

- a loop with changing bounds is usually a summation
- count one iteration per inner statement
- simplify the sum into a closed form

Example

$$\sum_{i=0}^{n-1} (n - i - 1) = \frac{n(n-1)}{2}$$

Pair Counting

- many quadratic algorithms count pairs
- the number of unordered pairs is $\binom{n}{2}$
- this equals $n(n-1)/2$

Interpretation

When you compare every element with every later element, expect $\Theta(n^2)$.

Why the First Few Terms Do Not Matter

- constants matter for benchmarking
- constants do not matter for asymptotic classification
- lower-order terms vanish after the threshold

Example

$1000000n + n^2$ is eventually dominated by n^2 .

Proof by Direct Inequality

To show $n \log n = O(n^{1.5})$:

- divide both sides by n
- compare $\log n$ with $\sqrt{n}$
- note that logarithms grow more slowly than any positive power

Conclusion

The polynomial term wins.

How to Negate a Claim

To disprove $f(n) = O(g(n))$:

- look at the ratio $f(n)/g(n)$
- if it becomes arbitrarily large, no constant works

Example

n^2 is not $O(n)$ because $n^2/n = n \rightarrow \infty$.

A Practical Proof Pattern

- 1 write the exact count
- 2 simplify the count
- 3 identify the dominant term
- 4 convert to O , Ω , or Θ

Why it helps

This makes the proof checkable by another person.

An Inductive View of Bounds

- induction is useful when a problem builds on smaller inputs
- it is common for recurrences and recursive algorithms
- we will use it more heavily in divide and conquer

Connection

Recurrence solving is the next natural step after asymptotic notation.

Fast Growth Rules

- if $a > 1$, then a^n beats every polynomial
- factorial growth beats exponentials
- logarithms are the slowest standard family we use often

Mental ladder

$$\log n \ll n \ll n \log n \ll n^2 \ll 2^n \ll n!$$

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison**
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

The Growth Ladder

Slowest	Fastest
$\log n$	$n!$
n	2^n
$n \log n$	n^2
$n^{1.5}$	3^n

Use

This ladder helps you compare algorithmic costs at a glance.

A Small Numerical Table

n	n	n^2	2^n
10	10	100	1024
20	20	400	1,048,576
30	30	900	1,073,741,824

- the gap between linear and exponential growth becomes huge quickly

Polynomial Versus Exponential

- polynomial-time algorithms usually scale
- exponential-time algorithms often fail beyond small n
- the cross-over is where practical algorithm design matters most

Important

An algorithm can be mathematically correct and still useless at scale.

Polynomial Versus Factorial

- factorial growth is worse than exponential growth
- brute-force permutation search is usually a dead end
- better structure or pruning is needed

Example

Scheduling by trying every order is rarely feasible.

Linear Versus Quadratic

- $1000n$ grows linearly
- n^2 grows quadratically
- linear eventually wins for large n

Break-even thinking

Ask where the two formulas are equal and which side is faster before and after that point.

Linear Versus Logarithmic

- binary search is faster than linear search on sorted data
- each comparison discards half the search space
- the number of rounds is about $\log_2 n$

Running example

A sorted student-ID list is the perfect place for binary search.

$n \log n$ in Context

- sorting by comparison often costs $n \log n$
- this is much better than n^2 for large n
- it is still more expensive than linear work

Typical use

Merge sort and heap-based methods often land here.

$n^{1.5}$ Versus n^2

- both are polynomial
- the smaller exponent matters a lot at scale
- even one-half power can change feasibility

Takeaway

Small improvements in the exponent are serious algorithmic improvements.

A Crossover Example

- algorithm A: $1000n$
- algorithm B: $n^2/1000$

Compare

For small n , the constant may favor one side. For large n , the lower-order growth wins.

When Small Inputs Mislead

- benchmark results on tiny data can hide the true growth class
- cache effects and interpreter overhead dominate early
- asymptotics describe the limit, not the first few data points

Rule

Do not infer the whole story from a small benchmark.

A Ranking Exercise

Rank from slowest growth to fastest growth:

$$\sqrt{n}, \log^2 n, n \log n, n^{1.5}, n^2, 2^n, 3^n, n!$$

Expected order

Logarithmic, root, linear-log, fractional polynomial, quadratic, exponential, factorial.

A Comparison Checklist

- compare the dominant terms
- ignore constants until the very end
- test with a ratio or a limit if needed
- remember that a larger exponent changes everything

Use this checklist on every growth comparison problem

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs**
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Linear Search Pseudocode

```
def linear_search(arr, target):  
    for i, x in enumerate(arr):  
        if x == target:  
            return i  
    return -1
```

- checks records from left to right
- best case: first element
- worst case: last element or absent target

Linear Search Analysis

- at most n comparisons in the worst case
- at least 1 comparison in the best case
- space usage is constant

Result

Worst-case time is $\Theta(n)$, extra space is $\Theta(1)$.

Binary Search Pseudocode

```
def binary_search(arr, target):  
    lo, hi = 0, len(arr) - 1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        if arr[mid] == target:  
            return mid  
        if arr[mid] < target:  
            lo = mid + 1  
        else:  
            hi = mid - 1  
    return -1
```

- requires sorted input
- halves the remaining interval on each step

Binary Search Analysis

- each step removes about half of the remaining candidates
- the number of rounds is bounded by $\lceil \log_2 n \rceil + 1$
- space usage is constant for the iterative version

Result

Worst-case time is $\Theta(\log n)$, extra space is $\Theta(1)$.

Insertion Sort Pseudocode

```
def insertion_sort(arr):  
    result = list(arr)  
    for j in range(1, len(result)):  
        key = result[j]  
        i = j - 1  
        while i >= 0 and result[i] > key:  
            result[i + 1] = result[i]  
            i -= 1  
        result[i + 1] = key  
    return result
```

- grows a sorted prefix one item at a time
- stable when the comparison is strict

Insertion Sort Walkthrough

- start with the first item as a sorted prefix
- take the next key
- shift larger elements right
- insert the key into the gap

Why it matters

Insertion sort is simple, stable, and good on nearly sorted input.

Insertion Sort Analysis

- best case: already sorted, about n work
- worst case: reverse sorted, about $n^2/2$ shifts
- average case: still quadratic

Conclusion

Insertion sort is $\Theta(n^2)$ in the worst case.

Min and Max by Pairs

- compare items in pairs first
- smaller item can only update the minimum
- larger item can only update the maximum

Benefit

This uses fewer comparisons than scanning separately for min and max.

Counting Nested Operations

- a nested loop often becomes a sum
- the inner loop count depends on the outer index
- simplify the sum carefully

Example

$$\sum_{i=0}^{n-1} (n - i - 1) = \frac{n(n-1)}{2}$$

Set-Based Pair Sum

- naive pair checking is quadratic
- a set can remember complements seen so far
- this often gives linear expected time

Pattern

Use extra space to reduce repeated searching.

Timing Experiments

- timing can show the trend, not prove the theorem
- repeat measurements to reduce noise
- compare the shape of the curve, not just one number

Important

Empirical timing supports analysis, but it does not replace analysis.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes**
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Decision Problems

- a decision problem asks yes/no
- complexity classes are usually defined for decision problems
- optimization problems are often converted into decision form

Example

"Is there a schedule with no conflicts?" is a decision problem.

Class P

- problems solvable in polynomial time
- examples: sorting, shortest path, bipartite checking
- this is the class of algorithms we generally consider efficient

Informal meaning

If the input grows by a factor of 10, polynomial-time algorithms usually remain manageable longer than exponential ones.

- solutions can be verified in polynomial time
- the certificate is a proposed solution
- the verifier checks it quickly

Example

For a schedule, the certificate could be the assignment of classes to rooms and times.

- P asks "can we solve it quickly?"
- NP asks "can we verify a proposed answer quickly?"
- many hard search problems fit this pattern

Key point

A fast verifier does not automatically give a fast solver.

NP-Complete

- in NP
- at least as hard as every problem in NP
- if one NP-complete problem has a polynomial-time algorithm, then every NP problem does

Importance

NP-complete problems are the central hard problems in classical complexity theory.

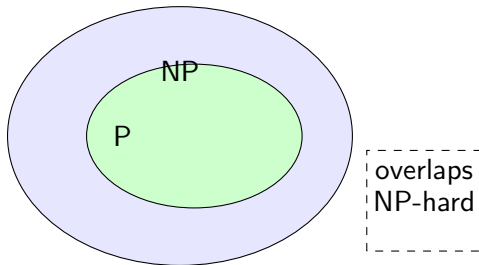
NP-Hard

- at least as hard as every problem in NP
- may not be a decision problem
- may not even be in NP

Example

Optimization versions of many NP-complete problems are NP-hard.

A Simple Inclusion Picture



- P sits inside NP
- NP-complete problems are the hard core inside NP

Famous Examples

Problem	Class
Sorting	P
Shortest path	P
2-colorability	P
3-SAT	NP-complete
Clique	NP-complete
Hamiltonian cycle	NP-complete
TSP decision version	NP-complete

Why Optimization Is Often Harder to State

- "find the best" usually turns into a search over many possibilities
- decision form often captures the same hardness more cleanly
- optimization can be NP-hard even when the decision version is NP-complete

Example

"Is there a schedule with at most k conflicts?" is easier to classify than "find the schedule with minimum conflicts."

The question

Does every problem whose answer can be checked quickly also have a quick algorithm for finding the answer?

- this is the famous P vs NP question
- it remains open
- most computer scientists believe $P \neq NP$

Practical Meaning of NP-Hardness

- do not expect a polynomial-time exact algorithm in the general case
- use heuristics, approximation, or special-case structure
- exploit domain constraints whenever possible

Engineering response

The right question is often "what kind of answer do we need, and how good does it need to be?"

A Campus Scheduling Example

- rooms have limited capacity
- classes cannot overlap for the same students
- teachers have availability constraints

Why this is hard

The number of feasible combinations grows very quickly.

Certificates and Verifiers in Practice

- a certificate is a candidate schedule
- the verifier checks room sizes, time clashes, and teacher constraints
- checking is much easier than searching

This is the NP pattern

Quick verification does not imply quick discovery.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions**
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

What Is a Reduction?

- transform instances of problem A into instances of problem B
- preserve the answer
- use B as a solver for A

Intuition

If A can be turned into B quickly, then B is at least as hard as A.

Reduction Template

- 1 start with an instance of A
- 2 encode it as an instance of B
- 3 prove the encoding is polynomial-time
- 4 prove yes instances map to yes instances and no instances map to no instances

Why Reductions Matter

- they let us compare problem hardness
- they let us reuse known hardness results
- they explain why many different problems share the same difficulty

In one sentence

Reductions are the bridge between new problems and known hard problems.

Example Reduction Pattern

- identify the hard structure inside the new problem
- map items to vertices, constraints to edges, or clauses to gadgets
- preserve satisfiability or feasibility

Common idea

Encode logical constraints into a combinatorial object.

Vertex Cover and Independent Set

- a vertex cover touches every edge
- an independent set contains no edge
- the two notions are complementary

Reduction idea

A graph has an independent set of size k if and only if it has a vertex cover of size $|V| - k$.

SAT as a Hardness Source

- Boolean satisfiability is the canonical NP-complete problem
- many reductions start from SAT or 3-SAT
- clauses become gadgets or structural constraints

Why SAT matters

Once SAT is hard, many other problems inherit hardness through reductions.

3-SAT and Clique

- variables and clauses can be encoded as graph vertices
- compatible choices become edges
- a clique represents a consistent selection

Conceptual takeaway

Reduction proofs are about preserving structure, not just translating words.

How to Read a Reduction Proof

- look for the source problem and the target problem
- identify the mapping
- check both directions of correctness
- check the time required for the transformation

Checklist

Source, target, transformation, correctness, complexity.

Common Reduction Mistakes

- proving only one direction
- forgetting the polynomial-time requirement
- confusing optimization with decision form
- assuming the reduction itself is the algorithm

Warning

A reduction does not solve the problem. It only relates it to another one.

Approximation and Heuristics

- exact solutions may be too expensive
- approximation algorithms give provable near-optimal answers
- heuristics give practical answers without a formal guarantee

Engineering lesson

For hard problems, good enough can be the right goal.

A Decision Version Example

- optimization: find the smallest number of rooms
- decision: is there a schedule using at most k rooms?
- the decision version is often the one used in complexity theory

Why

Decision problems fit clean yes/no verification.

Reduction Strategy for Exams

- ① identify a known NP-complete problem
- ② map its instances to the new problem
- ③ prove the mapping preserves yes/no answers
- ④ explain why the mapping is polynomial

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits**
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading

Decidable and Undecidable

- a decidable problem has an algorithm that always halts with the correct answer
- an undecidable problem has no such algorithm

Example

Connectivity is decidable. The halting problem is not.

The Halting Problem

Question

Given a program P and input I , will P eventually halt?

- Turing proved that no algorithm can solve this for all programs and inputs
- this is one of the foundational limits of computation

Why This Matters for Software

- there is no perfect general-purpose bug detector
- there is no general method that proves every program halts
- static analysis and testing are useful, but they are not omniscient

Practical meaning

Some questions are not just hard; they are impossible to solve in full generality.

Semi-Decidable Problems

- some problems have algorithms that halt on yes instances
- they may run forever on no instances
- this is weaker than decidability

Use

Semi-decidability appears in theorem proving and search over infinite spaces.

Diagonalisation Idea

- assume a perfect halting checker exists
- build a program that contradicts the checker on itself
- the contradiction shows the checker cannot exist

Method

This proof style is one of the classic tools for impossibility results.

Limits and Boundaries

- P, NP, NP-hard, and undecidable are different kinds of difficulty
- "hard" does not always mean the same thing
- choose the right language for the right limit

Summary

Some problems are slow. Some are search-hard. Some are impossible to decide.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs**
- 11 Advanced Problems and Further Reading

Exercise 2.1: Proofs by Definition

- $5n^2 + 3n = O(n^2)$
- $n^2/4 = \Omega(n)$
- $3n \log n + n = \Theta(n \log n)$
- $\log_2 n = \Theta(\log_{10} n)$

What to do

Write the constants explicitly and show the inequalities for large enough n .

Exercise 2.2: Ordering Growth Rates

- compare $\sqrt{n}$, $\log^2 n$, $n \log n$, $n^{1.5}$, n^2 , 2^n , 3^n , and $n!$
- use the growth ladder rather than raw calculation

Expected pattern

logarithmic, root, linear-log, fractional polynomial, quadratic, exponential, factorial

Exercise 2.3: Sums and Dominant Terms

- $\sum_{i=1}^n i^2$
- $\sum_{i=1}^{\log n} n$
- $\sum_{i=0}^n 2^i$
- $n^2 2^n + n^{100}$

Strategy

Use standard sum formulas and keep the dominant term.

Exercise 2.4: Properties

- $f(n) = O(g(n)) \Rightarrow g(n) = \Omega(f(n))$
- $f(n) + g(n) = \Theta(\max(f(n), g(n)))$
- if $f = O(g)$ and $g = O(h)$, then $fg = O(h^2)$

Use

These are algebraic tools, but every claim still needs justification.

Exercise 2.5: Loop Complexity

- nested doubling loops
- recursive split-and-combine recurrence
- unusual inner step sizes

What to look for

Count the number of iterations per outer loop and convert to a sum or recurrence.

Exercise 2.6: Space Complexity

- recursive factorial: stack frames grow with n
- iterative factorial: constant extra space
- recursive binary search: $O(\log n)$ stack space
- iterative binary search: $O(1)$ extra space

Exercise 2.7: Classify Problems

- sorting, shortest path, bipartite check: P
- 3-SAT, clique, Hamiltonian cycle: NP-complete
- graph coloring with 3 colors: NP-complete
- Eulerian circuit: P

Reminder

Always say whether you mean the decision or optimization version.

Exercise 2.8: The Halting Problem

- state the problem in terms of a program and input
- explain the diagonal argument at a high level
- connect the result to software limits

Exercise 2.9: Reductions in Your Own Words

- explain what it means to transform instances of one problem into another
- explain why a polynomial reduction preserves hardness

Core sentence

If a known hard problem reduces to a new problem, the new problem is at least as hard.

Challenge 2.10

- prove $\sum_{k=1}^n k = \Theta(n^2)$
- find explicit constants

Hint

Use the closed form $n(n+1)/2$ and compare it to n^2 .

Challenge 2.11

- if $f(n) = o(g(n))$, then $f(n) = O(g(n))$
- but then $f(n) \neq \Theta(g(n))$

Hint

The ratio $f(n)/g(n)$ is the key object.

Challenge 2.12

- integer sorting with van Emde Boas trees
- range size U matters
- compare the dependence on U with comparison sorting lower bounds

Research angle

The data structure can beat comparison sorting only when the key universe is structured enough.

Outline

- 1 Why This Week Matters
- 2 Big-O
- 3 Omega and Theta
- 4 Proof Toolkit
- 5 Growth Comparison
- 6 Algorithm Walkthroughs
- 7 Complexity Classes
- 8 Reductions
- 9 Undecidability and Limits
- 10 Exercise Walkthroughs
- 11 Advanced Problems and Further Reading**

Advanced Topic: Average Case Versus Worst Case

- worst-case analysis is robust
- average-case analysis depends on an input model
- randomized algorithms mix both ideas

Why it matters

A slow worst case does not always mean a bad practical algorithm, but the assumption must be explicit.

Advanced Topic: Fine-Grained Complexity

- not all polynomial algorithms are equally good
- n^2 and $n^{1.5}$ can be meaningfully different in practice
- fine-grained complexity studies the exact exponent and small improvements

Advanced Topic: Approximation

- exact optimization may be NP-hard
- approximation gives a provable near-optimal answer
- this is often the right tool for real systems

Campus example

A near-optimal exam schedule can be much better than no schedule at all.

Advanced Topic: Randomization

- randomness can simplify algorithms
- randomized quicksort is a classic example
- the analysis uses expected cost

Preview

Week 14 will return to this idea in more depth.

Practical Break-Even Points

- asymptotics explain the far end of the curve
- real systems also need a break-even point
- a slower asymptotic algorithm can win on tiny inputs

Engineering habit

Always ask both "what is the growth class?" and "where does the crossover happen?"

Quick Formula Sheet

- $f(n) = O(g(n))$: upper bound
- $f(n) = \Omega(g(n))$: lower bound
- $f(n) = \Theta(g(n))$: tight bound
- $f(n) = o(g(n))$: strictly smaller growth

One Worked Comparison

Compare these two costs:

$$A(n) = 3n^2 + 40n + 17, \quad B(n) = 1000n$$

- $A(n)$ is $\Theta(n^2)$
- $B(n)$ is $\Theta(n)$
- the linear algorithm wins eventually

Study Checklist

- can you state each definition from memory?
- can you prove one upper bound and one lower bound?
- can you rank the standard growth families?
- can you explain P, NP, NP-complete, and NP-hard in one sentence each?

Further Reading

- CLRS, Chapter 3 and Appendix on complexity
- Jeff Erickson, *Algorithms*
- Complexity Zoo: <https://complexityzoo.net/>
- MIT OCW 6.006 lectures on asymptotic analysis

Week 2 Summary

- 1 Big-O gives an upper bound
- 2 Big-Omega gives a lower bound
- 3 Big-Theta gives a tight bound
- 4 proofs need constants and a threshold
- 5 growth classes separate linear, polynomial, exponential, and factorial behavior
- 6 P, NP, NP-complete, NP-hard, and undecidable are different notions of difficulty
- 7 reductions are the main tool for comparing problem hardness

Next Week

- divide and conquer
- merge sort
- quick sort
- recurrence relations

Preparation

Review the growth ladder and the proof patterns before moving to recurrences.

Discussion Prompts

- Which part of algorithm analysis feels most mechanical?
- Which part feels easiest to get wrong?
- Which problem class would matter most in a student-information system?